

Setting up Kubernetes on ProxMox

Things to consider:

- If you don't understand how kubernetes work, I would advise reading documentation on the theory before moving onto the implementation
- This documentation will assume 1 master node and 3 worker nodes

What you need before starting

- 4 Ubuntu Server machine running with 4 gigabytes of ram and 32 gigabytes of disk space and 2 cores for the worker nodes and 4 cores for the master node

Step 1: Configure hostnames | ALL NODES

Part A:

In each of the machines modify the hostnames to something recognizable like below:

I changed the hostname of the master node to "k8s-master-node" by running the command:

```
sudo hostnamectl set-hostname "k8s-master-node"
```

I then changed the hostname of the 1st worker node to "k8s-worker-node-1" by running the command:

```
sudo hostnamectl set-hostname "k8s-worker-node-1"
```

- Do the same for the other 2 worker nodes

For the changes to go through run the following command on EVERY machine:

```
sudo exec bash
```

To verify the changes run the following command on EVERY machine:

```
hostname
```

Part B:

Update the /etc/host file and add the IP addresses and hostnames for the Master Node and worker nodes by running the following command:

```
sudo nano /etc/hosts
```

Once in the file change it like it is below but replace the example IP and example hostname to match the IP's and Hostnames of your machines of your machines you set in Step 1 Part A.

```

GNU nano 7.2
127.0.0.1 localhost
127.0.1.1 ccdc

192.168.1.211 k8s-master-node
192.168.1.189 k8s-worker-node-1
192.168.0.120 k8s-worker-node-2
192.168.0.121 k8s-worker-node-3

```

To ensure you can communicate to all the nodes in the cluster, from each node ping every machine using their hostname by running the following command:

```
ping -c 3 k8s-worker-node-1
```

The result should imitate this:

```

blueteam@k8s-master-node:~$ ping -c 3 k8s-worker-node-1
PING k8s-worker-node-1 (192.168.1.189) 56(84) bytes of data.
64 bytes from k8s-worker-node-1 (192.168.1.189): icmp_seq=1 ttl=64 time=0.465 ms
64 bytes from k8s-worker-node-1 (192.168.1.189): icmp_seq=2 ttl=64 time=0.329 ms
64 bytes from k8s-worker-node-1 (192.168.1.189): icmp_seq=3 ttl=64 time=0.338 ms

--- k8s-worker-node-1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2055ms
rtt min/avg/max/mdev = 0.329/0.377/0.465/0.062 ms
blueteam@k8s-master-node:~$

```

Step 2: Disable Swap space | ALL NODES

Part A:

To disable swap on the nodes, run the following command:

```
sudo swapoff -a
```

To persist the change run the following command to access & change the `/etc/fstab` file:

```
sudo nano /etc/fstab
```

In the file comment out the `'/swap.img'` line as shown below:

```

GNU nano 7.2 /etc/fstab *
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point> <type> <options> <dump> <pass>
# / was on /dev/ubuntu-vg/ubuntu-lv during curtin installation
/dev/disk/by-id/dm-uuid-LVM-dsNcMeknFNPYp1xf0ogJ4XTqLwgM6P6We21XysUpcmBnU2GHjWI701Ir39Qf2M2s / ext4 defaults 0 1
# /boot was on /dev/sda2 during curtin installation
/dev/disk/by-uuid/6774f94a-8703-4581-a3a0-b3a954d3f756 /boot ext4 defaults 0 1
#/swap.img none swap sw 0 0
# GOTTA COMMENT OUT THIS LINE TO DISABLE SWAP SPACE ~~~~~

```

To check if swap space has been disabled run the command:

```
swapon --show
```

If the output is blank, swap space has been disabled

Step 3: Load the Containerd modules | ALL NODES

Part A:

To enable and load modules run the following commands on ALL machines:

```
sudo modprobe overlay
```

```
sudo modprobe br_netfilter
```

Then create a configuration file to specify the modules to load them permanently using the following command on ALL machines:

```
**sudo tee /etc/modules-load.d/k8s.conf <<EOF
overlay
br_netfilter
EOF**
```

Step 4: Configure the IPv4 Networking | ALL NODES

Part A:

Create a K8 configuration file in the /etc/sysctl.d/ directory with the following command and then add the following lines after:

```
sudo nano /etc/sysctl.d/k8s.conf
```

Then add these lines in the k8s.conf file:

```
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
```

Apply these settings by running the following command:

```
sudo sysctl --system
```

Step 5: Installing Docker | ALL NODES

Part A:

To install docker from Ubuntu's default repos, run these command on ALL machines:

```
sudo apt update
sudo apt install docker.io -y
```

To verify if docker is running, run the following command on ALL machines:

```
sudo systemctl status docker
```

The output should look like this:

```
blueteam@k8s-master-node:~$ sudo systemctl status docker
[sudo] password for blueteam:
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-11-12 16:52:48 UTC; 3 days ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 21085 (dockerd)
      Tasks: 10
     Memory: 21.9M (peak: 22.7M)
        CPU: 38.835s
    CGroup: /system.slice/docker.service
            └─21085 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Nov 12 16:52:48 k8s-master-node dockerd[21085]: time="2025-11-12T16:52:48.312644919Z" level=info msg="API listen on /run/docker.sock"
Nov 12 16:52:48 k8s-master-node systemd[1]: Started docker.service - Docker Application Container Engine.
Nov 12 17:12:34 k8s-master-node dockerd[21085]: time="2025-11-12T17:12:34.409927810Z" level=error msg="Failed to get event" error="rpc error: code = Unavailable"
Nov 12 17:12:34 k8s-master-node dockerd[21085]: time="2025-11-12T17:12:34.410058514Z" level=info msg="Waiting for containerd to be ready to restart event process"
Nov 12 17:12:34 k8s-master-node dockerd[21085]: time="2025-11-12T17:12:34.409927804Z" level=error msg="Failed to get event" error="rpc error: code = Unavailable"
Nov 12 17:12:34 k8s-master-node dockerd[21085]: time="2025-11-12T17:12:34.410193004Z" level=info msg="Waiting for containerd to be ready to restart event process"
Nov 12 18:34:48 k8s-master-node dockerd[21085]: time="2025-11-12T18:34:48.640181979Z" level=error msg="Failed to get event" error="rpc error: code = Unavailable"
Nov 12 18:34:48 k8s-master-node dockerd[21085]: time="2025-11-12T18:34:48.640466333Z" level=info msg="Waiting for containerd to be ready to restart event process"
Nov 12 18:34:48 k8s-master-node dockerd[21085]: time="2025-11-12T18:34:48.640183189Z" level=error msg="Failed to get event" error="rpc error: code = Unavailable"
Nov 12 18:34:48 k8s-master-node dockerd[21085]: time="2025-11-12T18:34:48.640540899Z" level=info msg="Waiting for containerd to be ready to restart event process"
Hint: See 'man systemctl'.
```

If docker is not active run the following command:

```
sudo systemctl start docker
```

Also, enable the Docker daemon for AutoStart upon system startup with the following command:

```
sudo systemctl enable docker
```

Part B:

Now we need to configure containerd. We begin this by first creating the /etc/containerd directory by running the following command:

```
sudo mkdir /etc/containerd
```

Then create the default config file for containerd by running:

```
sudo sh -c "containerd config default > /etc/containerd/config.toml"
```

Then update the SystemdCgroup directive by setting it to true with the following command:

```
sudo sed -i 's/ SystemdCgroup = false/ SystemdCgroup = true/'
/etc/containerd/config.toml
```

Then restart containerd to apply the changes made by running:

```
sudo systemctl restart containerd.service
```

Then verify the containerd service is running correctly with the following command:

```
sudo systemctl status containerd.service
```

The output to look something like this:

```
blueteam@k8s-master-node:~$ sudo systemctl status containerd.service
* containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-11-12 18:34:48 UTC; 3 days ago
     Docs: https://containerd.io
   Main PID: 22481 (containerd)
      Tasks: 244
    Memory: 877.6M (peak: 1.5G)
       CPU: 1h 19min 7.776s
   CGroup: /system.slice/containerd.service
           └─22481 /usr/bin/containerd
               └─22741 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id be153f42680e6d08324e1dc8a4df15e4ac9bf276959992578542c41247151ac7 -address /run/cont
               └─22782 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 6757557d063db7ca27db95eacca400e98fb196c88bc7bf04d22f21e9869b04155 -address /run/cont
               └─22824 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 271e2a935d31b5d3f1af7eddb46c9d89515acb4ca7ae5fa5196d99bdc14e20a0f -address /run/cont
               └─22868 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id f52f3b6a1de9ef966c5eb05f1da09d66c5f46a6974eeec585ef78d7fd9ca88411 -address /run/cont
               └─23162 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 4858c8012a07d745b1801270c3850b00ee94072b6c906a0ce86993ae00ec1f64 -address /run/cont
               └─23440 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 2b91093bc2451332871988ca04f5230c247b07eaa715dd4c4a4520faef3304fd -address /run/cont
               └─23589 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 2e77739aa821946c427ff2612033b10e93c322c984264151d303fd184d5919db -address /run/cont
               └─23731 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 50d2fd3a1e5288fc7850582a75616c9481859573794f254e44a2790885e10307 -address /run/cont
               └─24614 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 12975daa9012139ec345edecb024e87acf9115785b486c063ab0bc15d04065fc -address /run/cont
               └─24874 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id e7c4db9c9cfbc20d2bdfc21f37b0a80cb1de9680c7a95303fa98a2c6e59a3f5842 -address /run/cont
               └─24917 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id f51e907286bc04590490c9ffabb213ff7f0d5525be3299f5bc7c849e59f2eb77 -address /run/cont
               └─24989 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 6cbe3da896c58b469004b3f985572eb4411b74453882f044ea1cfd199d431184 -address /run/cont
               └─25072 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 0111769614dd5899457b1fde9a5e3ec21320d88b3db3123bbe91b015674e5856 -address /run/cont
               └─25244 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 4d6dabb1ecc5e571dca3a5e665961e4985fa1f7637d50949cf97ea019afe2254 -address /run/cont
               └─25282 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id 39907a23c4656ff7e7dd26bb9364305f2478f092d9b99ad9794d337467beeded -address /run/cont
               └─26433 /usr/bin/containerd-shim-runc-v2 -namespace k8s.io -id d7d71804da7f4e344a801cfb427deec760b727e625e82964dd6ffed26c2deee3 -address /run/cont

Nov 15 08:14:49 k8s-master-node containerd[22481]: time="2025-11-15T08:14:49.328566815Z" level=info msg="StartContainer for \"834d4a539aa978c8d4bffc8d1dac2f50e
Nov 15 08:14:49 k8s-master-node containerd[22481]: time="2025-11-15T08:14:49.659101141Z" level=info msg="StartContainer for \"834d4a539aa978c8d4bffc8d1dac2f50e
Nov 15 08:15:12 k8s-master-node containerd[22481]: time="2025-11-15T08:15:12.035170716Z" level=info msg="CreateContainer within sandbox \"2b91093bc245133287198
Nov 15 08:15:16 k8s-master-node containerd[22481]: time="2025-11-15T08:15:16.122596841Z" level=info msg="CreateContainer within sandbox \"2b91093bc245133287198
Nov 15 08:15:16 k8s-master-node containerd[22481]: time="2025-11-15T08:15:16.123373193Z" level=info msg="StartContainer for \"7a2ae3c7f76da6b7355e139a53e7a8980
Nov 15 08:15:16 k8s-master-node containerd[22481]: time="2025-11-15T08:15:16.276153547Z" level=info msg="StartContainer for \"7a2ae3c7f76da6b7355e139a53e7a8980
Nov 15 08:15:19 k8s-master-node containerd[22481]: time="2025-11-15T08:15:19.035295776Z" level=info msg="CreateContainer within sandbox \"6757557d063db7ca27db9
Nov 15 08:15:19 k8s-master-node containerd[22481]: time="2025-11-15T08:15:19.238087243Z" level=info msg="CreateContainer within sandbox \"6757557d063db7ca27db9
Nov 15 08:15:19 k8s-master-node containerd[22481]: time="2025-11-15T08:15:19.299208044Z" level=info msg="StartContainer for \"402c6ef251a98ab7a98dae4c310678956
Nov 15 08:15:19 k8s-master-node containerd[22481]: time="2025-11-15T08:15:19.462002025Z" level=info msg="StartContainer for \"402c6ef251a98ab7a98dae4c310678956
```

If the containerd service was not running as shown above, then run the command below:

```
sudo systemctl start containerd
```

To get containerd to AutoStart upon system startup with the following command:

```
sudo systemctl enable containerd
```

Step 6: Install the components of Kubernetes | ALL NODES

Part A:

To begin, we need to install all prerequisite packages on all nodes by running:

```
sudo apt-get install curl ca-certificates apt-transport-https -y
```

Note:

- that as of now, we are on version 1.34 but this may change depending on when you read this documentation so make sure to check the latest version and change the next few commands accordingly

To add the Kubernetes GPG signing key run:

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --  
dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

Then, add the kubernetes repository on the machine by running:

```
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]  
https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /" | sudo tee  
/etc/apt/sources.list.d/kubernetes.list
```

After that, update the systems source list so it recognizes the new repo by running:

```
sudo apt update
```

Part B:

To install kubernetes we need 3 packages:

- kubeadm
- kubelet
- kubectl

To install all 3 components run the command:

```
sudo apt install kubelet kubeadm kubectl -y
```

Step 7: Initialize the Kubernetes cluster | ON JUST MASTER NODE

Part A:

We need to initialize the master node as the control plane and make a unique pod network for the cluster by running:

```
sudo kubeadm init --pod-network-cidr=10.125.0.0/16
```

After that is complete you will be given 3 commands to run in your output as shown below:

```
Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.1.211:6443 --token fypukf.hjr0dlo9y8qqovuy \
--discovery-token-ca-cert-hash sha256:aaa410fc712e4086810fe4d69c54dc0aa392d6a8033f3e3e615e
a4f19e29aee1
blueteam@k8s-master-node:~$
```

Run the first command to create a .kube directory in the home directory:

```
mkdir -p $HOME/.kube
```

Run the second command to copy the clusters config file into the .kube directory:

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

Run the third command to configure ownership of the copied config file so the user is allowed to use the config file to manage the cluster:

```
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Part B:

We need to now install the Calico service to secure communication between pods and external services. We can auto-assign IP's to pods for smooth communication

Note:

- **that as of now, we are on version 3.31.0 but this may change depending on when you read this documentation so make sure to check the latest version and change the commands accordingly**

Run the following command to deploy the Calico operator:

```
kubectl create -f  
https://raw.githubusercontent.com/projectcalico/calico/v3.31.0/manifests/tigera-  
operator.yaml
```

Then download Calicos custom-resources files by running:

```
curl  
https://raw.githubusercontent.com/projectcalico/calico/v3.31.0/manifests/custom-  
resources.yaml -O
```

Part C: Update calicos config to match the pod network

Now if you run 'ls' in the home directory you will see:

```
blueteam@k8s-master-node:~$ ls  
custom-resources.yaml  superVIKTORscript.sh  
blueteam@k8s-master-node:~$
```

We need to update the CIDR defined in this yaml file to match the pods network by running:

```
sed -i 's/cidr: 192\.\.168\.\.0\.\.0\./16/cidr: 10.125.0.0\./16/g' custom-resources.yaml
```

Then run the command:

```
kubectl create -f custom-resources.yaml
```

Part 8 | ALL NODES

Part A:

As of right now, the master node is the only node in the cluster, we can see this by running:

```
kubectl get nodes
```


This will show us:

```
[blueteam@k8s-master-node:~$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
k8s-master-node     NotReady control-plane 9m51s v1.34.2
blueteam@k8s-master-node:~$
```

We need to add the rest of the nodes on the cluster.

On the **MASTER NODE** run the following command to generate the command needed for our worker nodes to join the cluster:

```
sudo kubeadm token create --print-join-command
```

The output will look like:

```
[blueteam@k8s-master-node:~$ sudo kubeadm token create --print-join-command
kubeadm join 192.168.1.211:6443 --token 7g6dcb.2irbbghurkccit7w --discovery-token-ca-cert-hash sha256:aaa410fc712e4086810fe4d69c54dc0aa392d6a8033f3e3e615ea4f19e29aee1
```

Then on **ALL THE WORKER NODES** run the command given from the output above

```
kubeadm join 192.168.1.211:6443 --token 79gdcbl.2irbbghkurckcit7w --discovery-token-ca-cert-hash sha256:aaa410fc712e4086810fef4d69c54dc0aa392d6a8033f3e3e615ea4f19e29aee1
```

The output should look like below:

```
blueteam@k8s-worker-node-1:~$ sudo kubeadm join 192.168.1.211:6443 --token 7g6dcb.2irbbghurkccit7w --discovery-token-ca-cert-hash sha256:aaa410fc712e4086810fe4d69c54dc0aa392d6a8033f3e3e615ea4f19e29aee1
[sudo] password for blueteam:
[preflight] Running pre-flight checks
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"..
.
[preflight] Use 'kubeadm init phase upload-config kubeadm --config your-config-file' to re-upload it.
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/instance-config.yaml"
[patches] Applied patch of type "application/strategic-merge-patch+json" to target "kubeletconfiguration"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.503100655s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

Now on **THE MASTER NODE** run the command below to verify all the worker nodes have joined:

```
kubectl get nodes
```


The output should look similar to the example below:

```
blueteam@k8s-master-node:~$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
k8s-master-node     Ready    control-plane   3d5h   v1.34.2
k8s-worker-node-1    Ready    <none>         3d5h   v1.34.2
k8s-worker-node-2    Ready    <none>         3d5h   v1.34.2
k8s-worker-node-3    Ready    <none>         3d5h   v1.34.2
blueteam@k8s-master-node:~$
```

To check the pods in all the namespaces run the command:

```
kubectl get pods -A
```

Step 9

Part A:

We will now test if the cluster is functional by deploying a service, we will go with nginx

In the **MASTER NODE** create a namespace to isolate the deployment, we will name it demo-namespace by running:

```
kubectl create namespace demo-namespace
```

Then we will create a deployment inside the namespace, specify the nginx image name and 2 replicates. We will call the deployment "testing-nginx". To do all this run:

```
kubectl create deployment testing-nginx --image nginx --replicas 2 --namespace demo-namespace
```

To verify the deployment in the namespace run the command:

```
kubectl get deployment -n demo-namespace
```

The output should show "testing-nginx" as shown below:

```
blueteam@k8s-master-node:~$ kubectl get deployment -n demo-namespace
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
my-app        2/2     2             2           3d5h
testing-nginx 2/2     2             2           3d5h
blueteam@k8s-master-node:~$
```

We will now expose the deployment using the NodePort service and set it up to run an application on port 80 for this example. To do this run:

```
kubectl expose deployment testing-nginx -n demo-namespace --type NodePort --port 80
```

Now to list the services in the namespace run:

```
kubectl get svc -n demo-namespace
```

The output should confirm NodePort running on the namespace as shown in the example below:

```
blueteam@k8s-master-node:~$ kubectl get svc -n demo-namespace
NAME          TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
testing-nginx  NodePort    10.104.32.248 <none>       80:32153/TCP     3d5h
blueteam@k8s-master-node:~$
```

Note this output tells us the node-port IP for nginx is 32153 which we will need later

To test if we can actually access the web-application, from any of **THE WORKER NODES** curl the web page by running the command:

```
curl http://<WORKER-IP>:<NODE PORT>
```

The output should look similar to below:

```
blueteam@k8s-master-node:~$ curl 192.168.0.120:32153
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
blueteam@k8s-master-node:~$
```

We now have nginx running successfully on the cluster!!

Thats it, thats a functioning k8 cluster right there.